TRABALHO EM GRUPO — ESTRUTURA DE DADOS

Pilha Estática em Python: Sistema de Controle de Documentos

Disciplina: Estrutura de Dados
Tema: Pilha Estática
Linguagem: Python
Integrantes: Grupos de 1 a 5 alunos

1. Contextualização

Imagine um setor administrativo que recebe documentos para análise. Os documentos são empilhados em uma estrutura, e o funcionário deve trabalhar sempre com o documento que está no topo.

O sistema precisa permitir inserir, retirar, consultar e localizar documentos. Entretanto, existe uma limitação: a pilha possui uma **capacidade máxima de 5 documentos**.

Sua equipe deverá desenvolver um programa em Python que simule esse sistema utilizando uma **pilha estática implementada manualmente**.

2. Regras obrigatórias

A implementação deverá seguir estas regras:

A pilha deverá ter capacidade fixa de 5 documentos.

A estrutura deverá ser criada com uma lista previamente dimensionada, por exemplo: pilha = [None] * 5.

O controle do topo deverá utilizar uma variável inteira, iniciando em -1.

O programa deverá ser dividido em funções.

Não será permitido utilizar append() e pop() para implementar as operações principais da pilha.

Não será permitido aumentar a capacidade da pilha durante a execução.

O programa não poderá encerrar inesperadamente quando o usuário digitar uma opção inválida.

O sistema deverá impedir inserções quando a pilha estiver cheia e remoções quando estiver vazia.

Atenção: usar uma lista do Python é permitido. O desafio é implementar manualmente o comportamento da pilha, controlando as posições por meio de índices.

3. Funcionalidades obrigatórias

Opção 1 — Inserir documento (PUSH)

Solicitar ao usuário o código e a descrição do documento.

O documento deverá ser inserido na próxima posição disponível da pilha, e o topo deverá ser atualizado.

Se a pilha estiver cheia, exibir:

ERRO: pilha cheia. Não é possível inserir outro documento.

Opção 2 — Retirar documento (POP)

Retirar o documento que estiver no topo da pilha e apresentar seus dados na tela.
Após a retirada, o topo deverá ser atualizado.
Se a pilha estiver vazia, exibir:
ERRO: pilha vazia. Não há documento para retirar.

Opção 3 — Consultar o documento do topo (PEEK)

Mostrar o código e a descrição do documento no topo, sem removê-lo.
A consulta não poderá modificar a pilha.

Opção 4 — Exibir todos os documentos

Mostrar os documentos do topo para a base.
Exemplo:
TOPO
Requerimento
Declaração
[101] Contrato
BASE
Se a pilha estiver vazia, informar que não há documentos armazenados.

Opção 5 — Pesquisar documento por código

Solicitar um código e procurar o documento na pilha.
Se encontrar, mostrar sua descrição e a posição em que está armazenado.
Se não encontrar, informar que o documento não existe na pilha.
A pesquisa não poderá remover nem alterar documentos.

Opção 6 — Exibir estatísticas

Exemplo:
Capacidade máxima: 5
Documentos armazenados: 3
Posições livres: 2
Ocupação: 60%

Opção 0 — Encerrar o programa

Finalizar somente quando o usuário escolher a opção zero.

4. Organização sugerida do programa

O código deverá possuir, no mínimo, as seguintes funções:

Função	Responsabilidade
inserir_documento()	Inserir documento no topo
retirar_documento()	Remover documento do topo
consultar_topo()	Consultar sem remover
exibir_pilha()	Mostrar os documentos
pesquisar_documento()	Localizar por código
exibir_estatisticas()	Calcular ocupação e posições livres
exibir_menu()	Apresentar as opções

A equipe poderá criar outras funções para melhorar a organização.

5. Estrutura de dados

Cada documento poderá ser representado por um dicionário:

```
documento = {
        "codigo": 101,
        "descricao": "Contrato de estagio"
}
```

A pilha deverá ter uma estrutura semelhante a esta:

```
CAPACIDADE = 5
pilha = [None] * CAPACIDADE
topo = -1
```

Essas linhas são apenas o ponto de partida. A equipe deverá implementar as operações e o menu completo.

6. Conceitos que precisam ser compreendidos

Como funciona o topo?

O topo indica a posição do último elemento inserido.

Quando topo == -1, a pilha está vazia.

Quando topo == CAPACIDADE - 1, a pilha está cheia.

Para inserir, primeiro verifique se há espaço.

Para retirar, primeiro verifique se existe algum elemento.

Exemplo de evolução da pilha

Operação	Topo	Conteúdo
Inicialização	-1	Vazia
Inserir documento A	0	A
Inserir documento B	1	A, B
Inserir documento C	2	A, B, C
Retirar documento	1	A, B
Inserir documento D	2	A, B, D

Observe que, após inserir A, B e C, a retirada remove C, pois ele está no topo. Esse é o comportamento LIFO.

7. Testes obrigatórios

Antes de entregar o trabalho, a equipe deverá testar as seguintes situações:

Consultar o topo com a pilha vazia.

Retirar um documento com a pilha vazia.

Inserir um documento e consultar o topo.

Inserir cinco documentos.

Tentar inserir um sexto documento.

Retirar um documento e verificar se o topo foi atualizado.

Pesquisar um código existente.

Pesquisar um código inexistente.

Exibir as estatísticas com a pilha parcialmente preenchida.

Esvaziar a pilha e verificar se o programa continua funcionando.

Digitar uma opção inválida no menu.

A equipe deverá registrar o resultado esperado e o resultado observado de cada teste.

8. Desafios extras

Impedir códigos duplicados na pilha.

Exibir um alerta quando a pilha estiver quase cheia.

Permitir limpar toda a pilha após confirmação do usuário.

Exibir a quantidade de documentos cuja descrição contenha uma palavra informada pelo usuário.

O recurso extra deverá respeitar a capacidade fixa e as regras da pilha estática.

9. Entregáveis

Cada grupo deverá entregar:

1.Arquivo Python (.py) com o programa completo.

2. Relatório curto explicando como o topo é controlado.

3. Tabela com os testes executados e seus resultados.

4. Capturas de tela demonstrando inserção, remoção, pesquisa, estatísticas e tratamento de erros.